

SIMATS ENGINEERING

ASSESSMENT TOOL 2 – LABORATORY PERFORMANCE QUESTIONS

COURSE CODE: CSA0904

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO4: *Applet Programming*-Applet Architecture - Simple Applet Display Methods
Event handling in Java - Delegation Event Model - Event Classes - Sources of Events - Event
Listener Interfaces -AWT Component and GUI Design : AWT Classes, Controls, Layout
Managers, and Menus.(BL3,BL4)
Assessment Tool Weightage: 20%

QUESTIONS: 2

Total Marks: 100

ANNEXURE A

LABORATORY PERFORMANCE QUESTIONS

Code of Conduct:

***I O. Bhanu Prakash reddy-192372075 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.***

Signature of the student: obhanuprakashreddy

JAVA PROGRAMMING

LABORATORY PERFORMANCE QUESTIONS

1. Develop a simple Applet that displays your name, roll number, and course in different colors and fonts. Use the paint() method for rendering.

Code:

```
import java.applet.Applet;
import java.awt.*;

public class StudentApplet extends Applet {

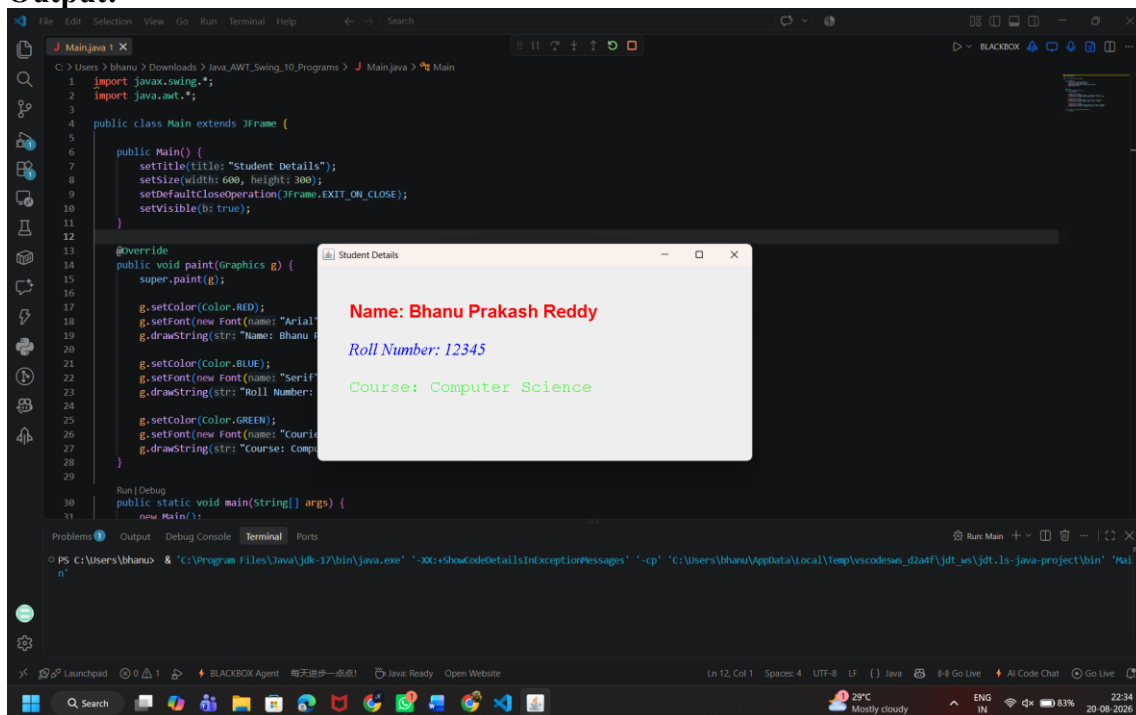
    public void paint(Graphics g) {

        g.setColor(Color.RED);
        g.setFont(new Font("Arial", Font.BOLD, 24));
        g.drawString("Name: Bhanu Prakash Reddy", 50, 80);

        g.setColor(Color.BLUE);
        g.setFont(new Font("Serif", Font.ITALIC, 22));
        g.drawString("Roll Number: 12345", 50, 130);

        g.setColor(Color.GREEN);
        g.setFont(new Font("Courier New", Font.PLAIN, 22));
        g.drawString("Course: Computer Science", 50, 180);
    }
}
```

Output:



2. Create two types of Applets: a) Stand-alone Applet b) HTML embedded Applet. Demonstrate both and explain the differences.

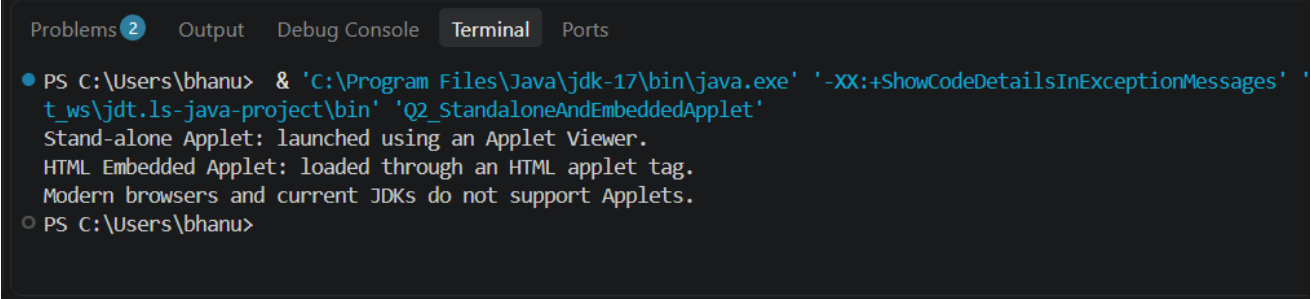
Code:

```
import java.applet.Applet;
import java.awt.*;

public class Q2_StandaloneAndEmbeddedApplet extends Applet {
    public void paint(Graphics g) {
        g.setColor(Color.BLUE);
        g.drawString("HTML Embedded Applet", 50, 80);
        g.drawString("The same Applet class can be used with an Applet Viewer.", 50, 120);
    }

    public static void main(String[] args) {
        System.out.println("Stand-alone Applet: launched using an Applet Viewer.");
        System.out.println("HTML Embedded Applet: loaded through an HTML applet tag.");
        System.out.println("Modern browsers and current JDKs do not support Applets.");
    }
}
```

Output:



```
Problems 2 Output Debug Console Terminal Ports
● PS C:\Users\bhanu> & 'C:\Program Files\Java\jdk-17\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' 't_ws\jdt.ls-java-project\bin' 'Q2_StandaloneAndEmbeddedApplet'
Stand-alone Applet: launched using an Applet Viewer.
HTML Embedded Applet: loaded through an HTML applet tag.
Modern browsers and current JDKs do not support Applets.
○ PS C:\Users\bhanu>
```

- 3. Design an Applet that accepts a number from the user and displays its square when a button is clicked. Use AWT controls and appropriate event handling.**

Code:

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class Q3_SquareApplet extends Applet implements ActionListener {
    TextField numberField;
    Button button;
    Label resultLabel;

    public void init() {
        add(new Label("Enter Number:"));
        numberField = new TextField(10);
        add(numberField);

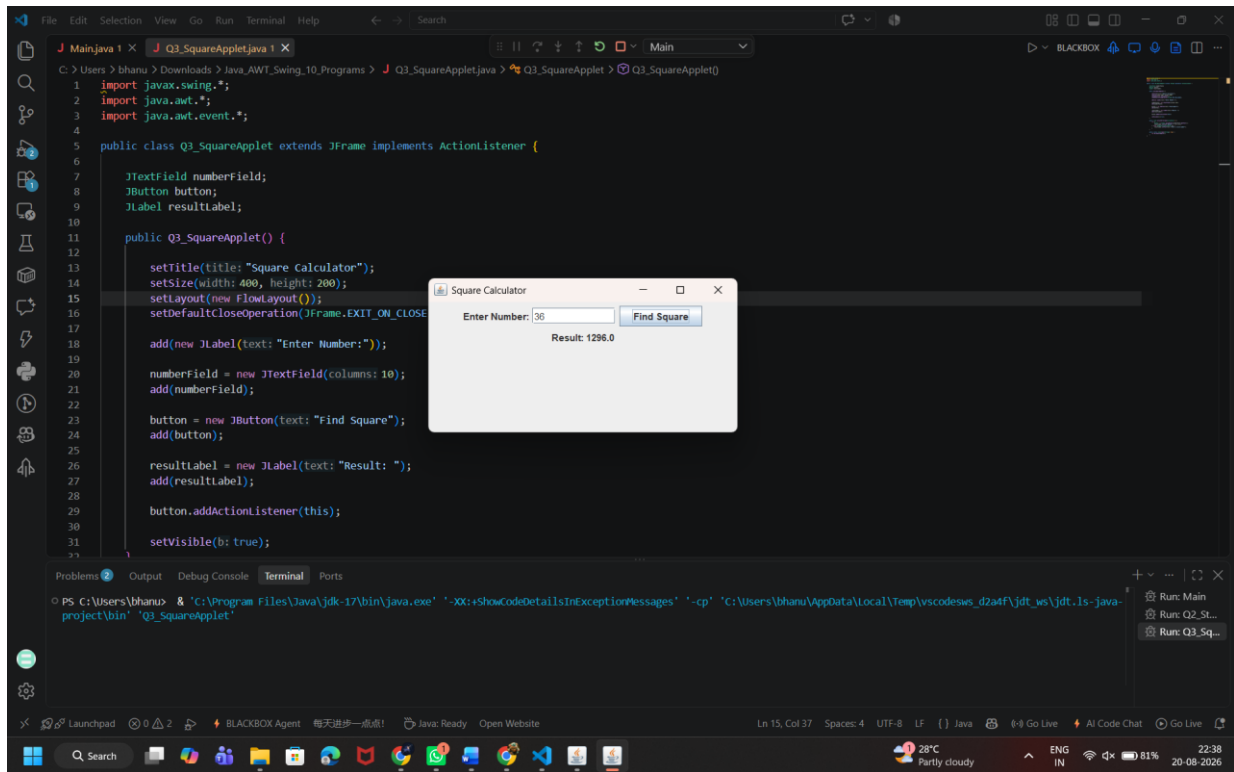
        button = new Button("Find Square");
        add(button);

        resultLabel = new Label("Result: ");
        add(resultLabel);

        button.addActionListener(this);
    }

    public void actionPerformed(ActionEvent e) {
        try {
            double n = Double.parseDouble(numberField.getText());
            resultLabel.setText("Result: " + (n * n));
        } catch (NumberFormatException ex) {
            resultLabel.setText("Enter a valid number");
        }
    }
}
```

Output:



- 4. Develop a Swing application that demonstrates the use of the following layout managers: FlowLayout, BorderLayout, and GridLayout. Switch among layouts using radio buttons.**

Code:

```
import javax.swing.*;
import java.awt.*;

public class Q4_LayoutManagerDemo extends JFrame {
    JPanel panel;

    Q4_LayoutManagerDemo() {
        setTitle("Layout Manager Demo");
        setSize(600, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JPanel top = new JPanel();
        JRadioButton flow = new JRadioButton("FlowLayout", true);
        JRadioButton border = new JRadioButton("BorderLayout");
        JRadioButton grid = new JRadioButton("GridLayout");

        ButtonGroup group = new ButtonGroup();
        group.add(flow);
        group.add(border);
        group.add(grid);

        top.add(flow);
        top.add(border);
        top.add(grid);
        add(top, BorderLayout.NORTH);

        panel = new JPanel();
        add(panel, BorderLayout.CENTER);

        flow.addActionListener(e -> changeLayout("flow"));
        border.addActionListener(e -> changeLayout("border"));
        grid.addActionListener(e -> changeLayout("grid"));

        changeLayout("flow");
    }

    void changeLayout(String type) {
        panel.removeAll();

        if (type.equals("flow")) {
            panel.setLayout(new FlowLayout());
            for (int i = 1; i <= 5; i++) panel.add(new JButton("Button " + i));
        } else if (type.equals("border")) {
            panel.setLayout(new BorderLayout());
            panel.add(new JButton("North"), BorderLayout.NORTH);
            panel.add(new JButton("South"), BorderLayout.SOUTH);
        }
    }
}
```

```

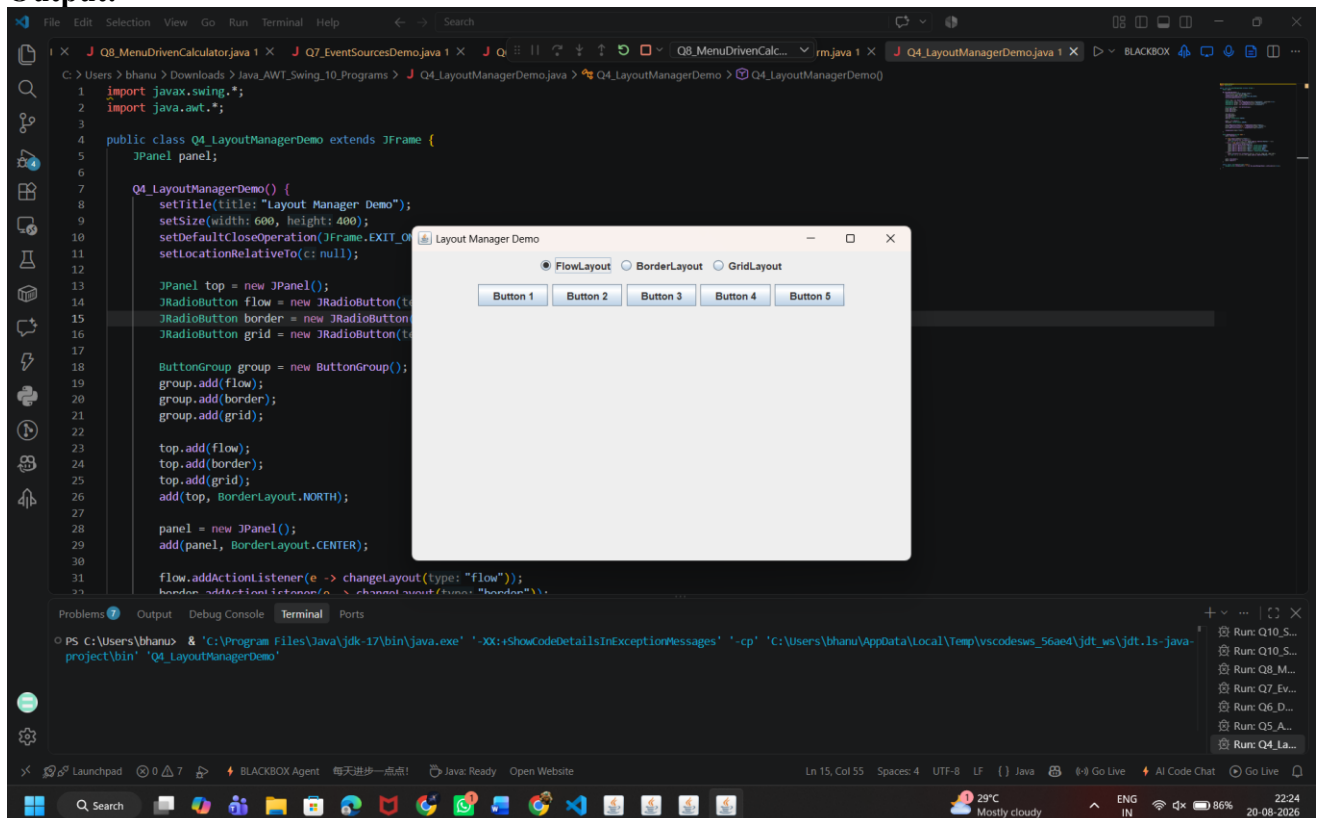
        panel.add(new JButton("East"), BorderLayout.EAST);
        panel.add(new JButton("West"), BorderLayout.WEST);
        panel.add(new JButton("Center"), BorderLayout.CENTER);
    } else {
        panel.setLayout(new GridLayout(2, 3, 10, 10));
        for (int i = 1; i <= 6; i++) panel.add(new JButton("Button " + i));
    }

    panel.revalidate();
    panel.repaint();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new Q4_LayoutManagerDemo().setVisible(true));
}
}

```

Output:



- 5. Create a GUI using AWT controls (Button, Label, TextField, Checkbox, Choice).
Perform simple operations such as registration form input and display the entered data.**

Code:

```
import java.awt.*;
import java.awt.event.*;

public class Q5_AWTRegistrationForm extends Frame implements ActionListener {
    TextField nameField, emailField;
    Checkbox male, female;
    Choice courseChoice;
    Button submitButton;
    TextArea output;

    Q5_AWTRegistrationForm() {
        setTitle("AWT Registration Form");
        setSize(500, 500);
        setLayout(new GridLayout(6, 2, 10, 10));

        add(new Label("Name:"));
        nameField = new TextField();
        add(nameField);

        add(new Label("Email:"));
        emailField = new TextField();
        add(emailField);

        add(new Label("Gender:"));
        Panel p = new Panel();
        CheckboxGroup group = new CheckboxGroup();
        male = new Checkbox("Male", group, true);
        female = new Checkbox("Female", group, false);
        p.add(male);
        p.add(female);
        add(p);

        add(new Label("Course:"));
        courseChoice = new Choice();
        courseChoice.add("Computer Science");
        courseChoice.add("Information Technology");
        courseChoice.add("Electronics");
        courseChoice.add("Mechanical");
        add(courseChoice);

        submitButton = new Button("Submit");
        add(submitButton);

        output = new TextArea();
        add(output);

        submitButton.addActionListener(this);
    }
}
```



```

addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) { dispose(); }
});

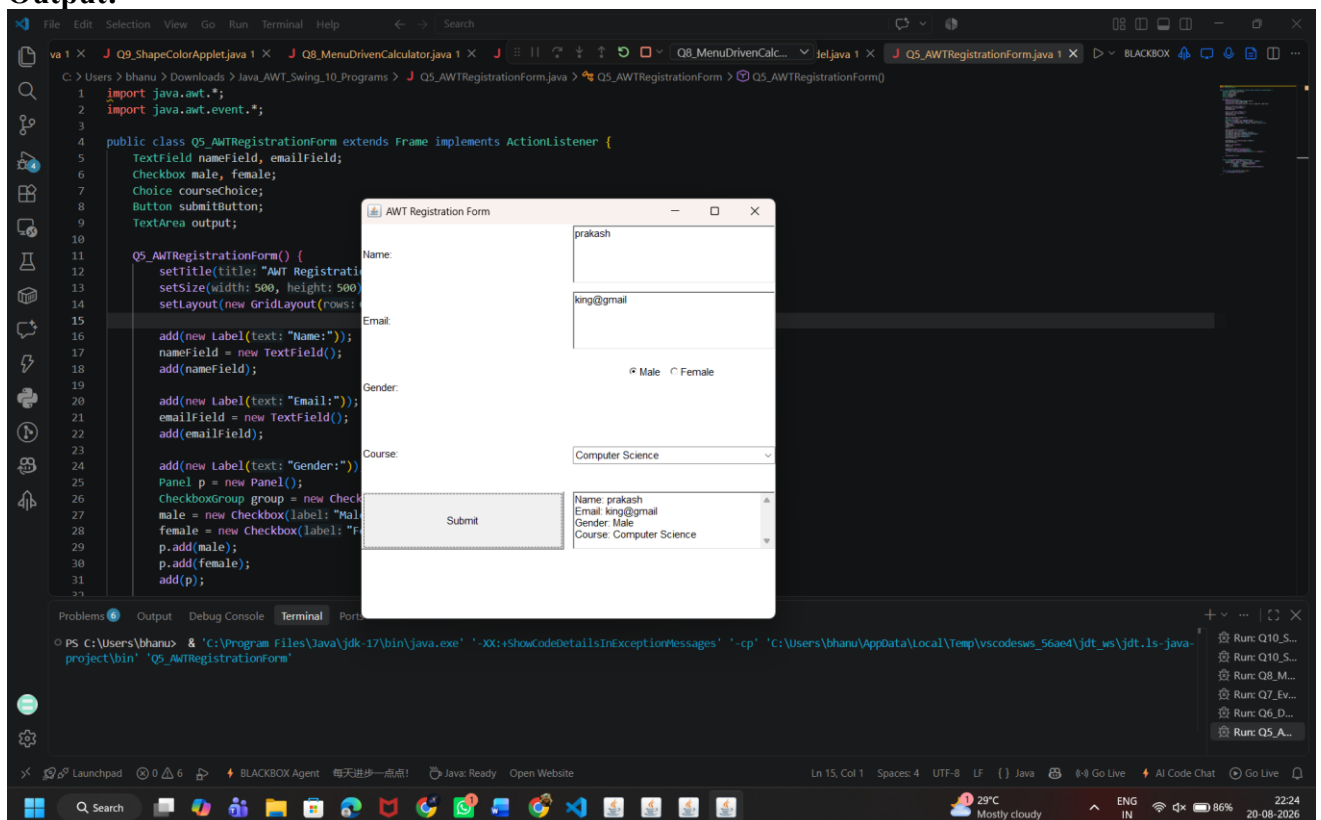
setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    String gender = male.getState() ? "Male" : "Female";
    output.setText("Name: " + nameField.getText()
        + "\nEmail: " + emailField.getText()
        + "\nGender: " + gender
        + "\nCourse: " + courseChoice.getSelectedItem());
}

public static void main(String[] args) {
    new Q5_AWTRegistrationForm();
}
}

```

Output:



6. Develop a program that demonstrates event handling using the Delegation Event Model. Handle at least two types of events (ActionEvent, ItemEvent) using listener interfaces.

Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Q6_DelegationEventModel extends JFrame implements ActionListener, ItemListener {
    JLabel label = new JLabel("Event: ");
    JButton button = new JButton("Click Button");
    JCheckBox checkBox = new JCheckBox("Select Checkbox");

    Q6_DelegationEventModel() {
        setTitle("Delegation Event Model");
        setSize(500, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());

        add(button);
        add(checkBox);
        add(label);

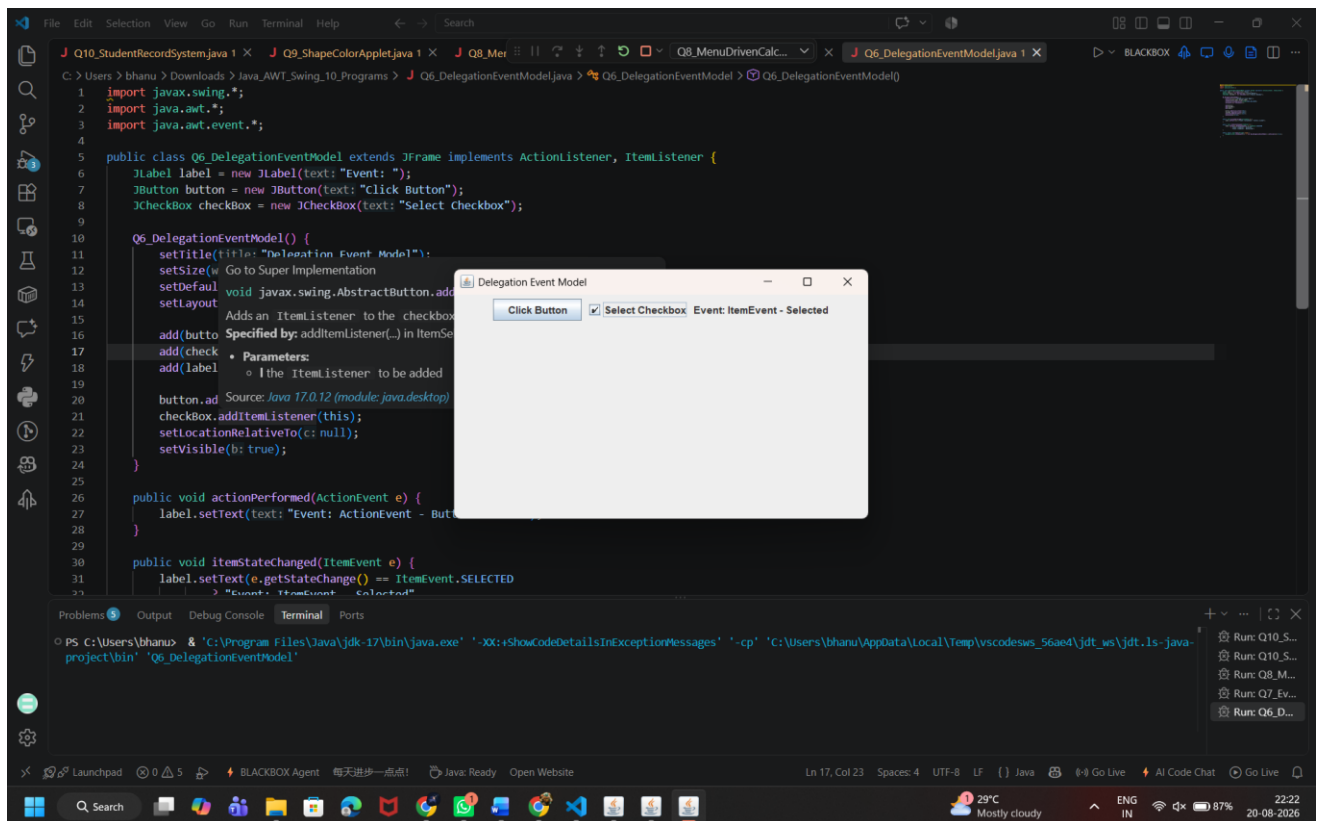
        button.addActionListener(this);
        checkBox.addItemListener(this);
        setLocationRelativeTo(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        label.setText("Event:(ActionEvent - Button clicked");
    }

    public void itemStateChanged(ItemEvent e) {
        label.setText(e.getStateChange() == ItemEvent.SELECTED
            ? "Event: ItemEvent - Selected"
            : "Event: ItemEvent - Deselected");
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Q6_DelegationEventModel().setVisible(true));
    }
}
```

Output:



7. Write a program that demonstrates different sources of events (Button click, TextField input, Checkbox selection, Window closing). Display the event source and type.

Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Q7_EventSourcesDemo extends JFrame
    implements ActionListener, ItemListener, WindowListener {
    JButton button = new JButton("Button Click");
    JTextField textField = new JTextField(15);
    JCheckBox checkBox = new JCheckBox("Checkbox");
    JTextArea output = new JTextArea(15, 45);

    Q7_EventSourcesDemo() {
        setTitle("Event Sources Demo");
        setSize(600, 400);
        setLayout(new FlowLayout());

        output.setEditable(false);
        add(button);
        add(textField);
        add(checkBox);
        add(new JScrollPane(output));

        button.addActionListener(this);
        textField.addActionListener(this);
        checkBox.addItemListener(this);
        addWindowListener(this);

        setLocationRelativeTo(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        output.append(e.getSource() == button
            ? "Source: Button | Type: ActionEvent\n"
            : "Source: TextField | Type: ActionEvent\n");
    }

    public void itemStateChanged(ItemEvent e) {
        output.append("Source: Checkbox | Type: ItemEvent\n");
    }

    public void windowClosing(WindowEvent e) {
        output.append("Source: Window | Type: WindowEvent - Closing\n");
        dispose();
    }

    public void windowOpened(WindowEvent e) {}
}
```

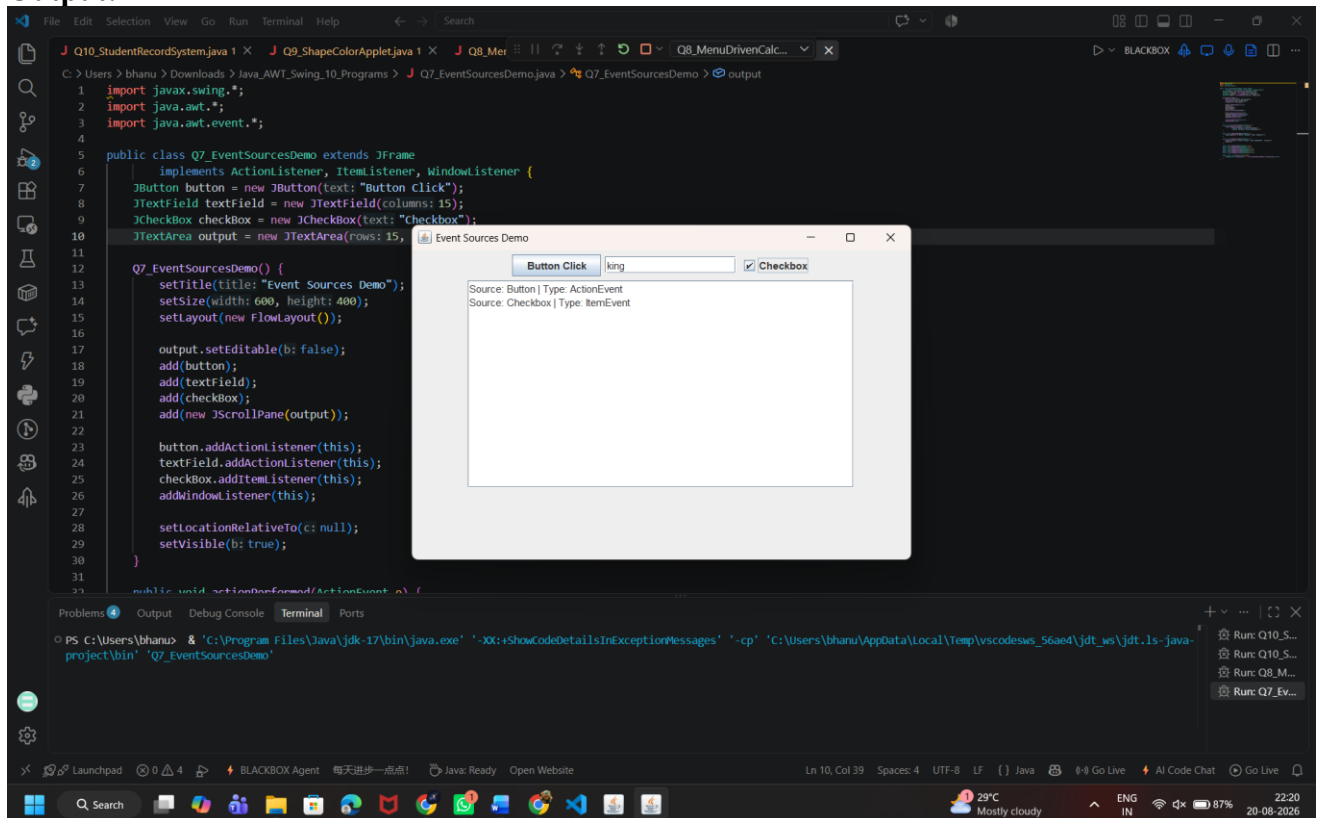
```

public void windowClosed(WindowEvent e) {}
public void windowIconified(WindowEvent e) {}
public void windowDeiconified(WindowEvent e) {}
public void windowActivated(WindowEvent e) {}
public void windowDeactivated(WindowEvent e) {}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new Q7_EventSourcesDemo().setVisible(true));
}
}

```

Output:



8. Create a menu-driven Swing application for a simple calculator (Add, Subtract, Multiply, Divide, Clear, Exit) using MenuBar, Menu, and MenuItem.

Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Q8_MenuDrivenCalculator extends JFrame implements ActionListener {
    JTextField num1 = new JTextField();
    JTextField num2 = new JTextField();
    JTextField result = new JTextField();

    JMenuItem add, subtract, multiply, divide, clear, exit;

    Q8_MenuDrivenCalculator() {
        setTitle("Menu Driven Calculator");
        setSize(500, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new GridLayout(3, 2, 10, 10));

        result.setEditable(false);
        add(new JLabel("Number 1:")); add(num1);
        add(new JLabel("Number 2:")); add(num2);
        add(new JLabel("Result:")); add(result);

        JMenuBar bar = new JMenuBar();
        JMenu menu = new JMenu("Calculator");

        add = new JMenuItem("Add");
        subtract = new JMenuItem("Subtract");
        multiply = new JMenuItem("Multiply");
        divide = new JMenuItem("Divide");
        clear = new JMenuItem("Clear");
        exit = new JMenuItem("Exit");

        menu.add(add); menu.add(subtract); menu.add(multiply);
        menu.add(divide); menu.addSeparator(); menu.add(clear); menu.add(exit);
        bar.add(menu);
        setJMenuBar(bar);

        add.addActionListener(this);
        subtract.addActionListener(this);
        multiply.addActionListener(this);
        divide.addActionListener(this);
        clear.addActionListener(this);
        exit.addActionListener(this);

        setLocationRelativeTo(null);
    }
}
```

```

        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == exit) {
            System.exit(0);
        }

        if (e.getSource() == clear) {
            num1.setText(""); num2.setText(""); result.setText("");
            return;
        }

        try {
            double a = Double.parseDouble(num1.getText());
            double b = Double.parseDouble(num2.getText());
            double ans;

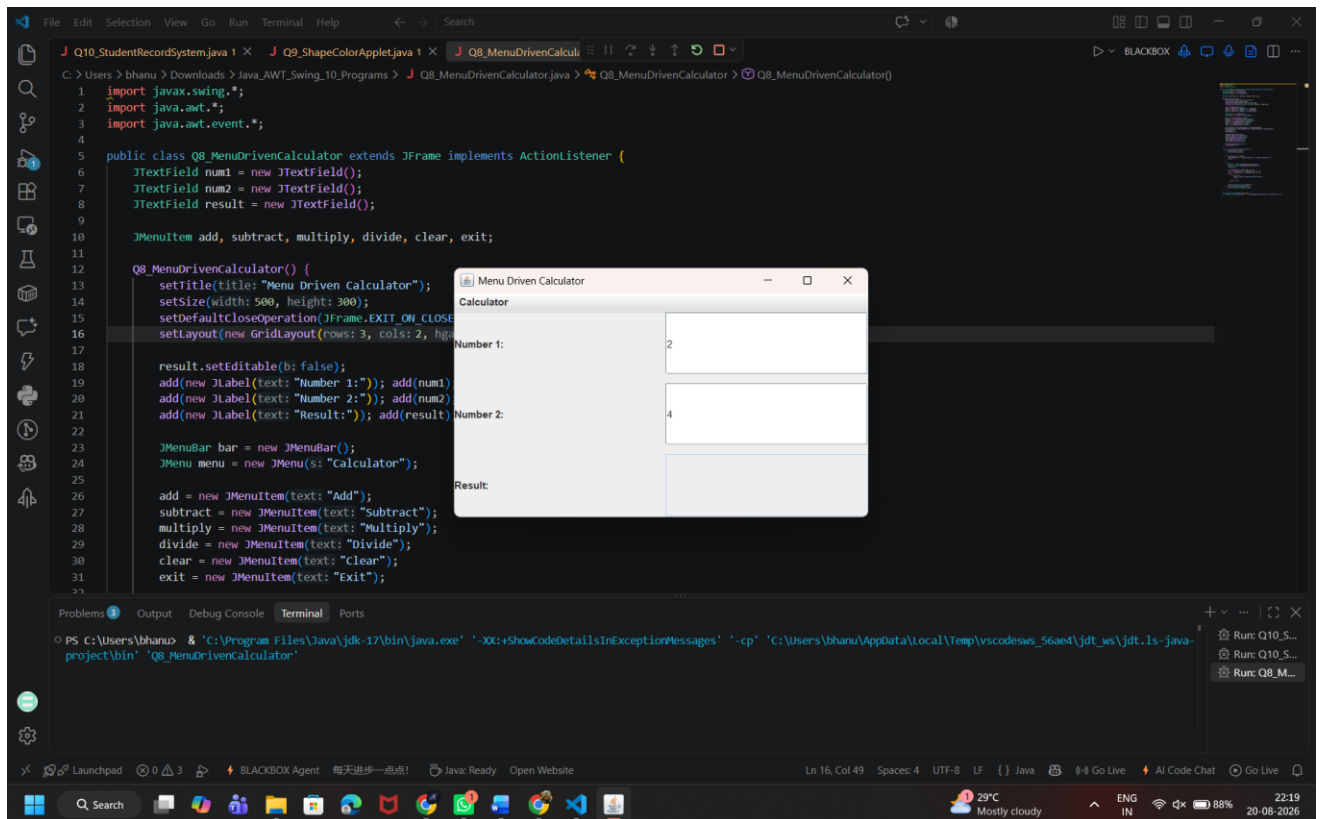
            if (e.getSource() == add) ans = a + b;
            else if (e.getSource() == subtract) ans = a - b;
            else if (e.getSource() == multiply) ans = a * b;
            else {
                if (b == 0) {
                    result.setText("Cannot divide by zero");
                    return;
                }
                ans = a / b;
            }

            result.setText(String.valueOf(ans));
        } catch (NumberFormatException ex) {
            result.setText("Invalid input");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Q8_MenuDrivenCalculator().setVisible(true));
    }
}

```

Output:



9. Develop an Applet that draws basic shapes (line, rectangle, oval) and changes their color based on user selection from a drop-down list (Choice control).

Code:

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class Q4_ShapesApplet extends Applet implements ItemListener {

    Choice shapeChoice;
    Choice colorChoice;

    public void init() {
        shapeChoice = new Choice();
        shapeChoice.add("Line");
        shapeChoice.add("Rectangle");
        shapeChoice.add("Oval");

        colorChoice = new Choice();
        colorChoice.add("Red");
        colorChoice.add("Blue");
        colorChoice.add("Green");
        colorChoice.add("Black");

        add(new Label("Select Shape:"));
        add(shapeChoice);

        add(new Label("Select Color:"));
        add(colorChoice);

        shapeChoice.addItemListener(this);
        colorChoice.addItemListener(this);
    }

    public void itemStateChanged(ItemEvent e) {
        repaint();
    }

    public void paint(Graphics g) {

        String shape = shapeChoice.getSelectedItem();
        String color = colorChoice.getSelectedItem();

        if (color.equals("Red"))
            g.setColor(Color.RED);
        else if (color.equals("Blue"))
            g.setColor(Color.BLUE);
        else if (color.equals("Green"))
            g.setColor(Color.GREEN);
        else
```

```

        g.setColor(Color.BLACK);

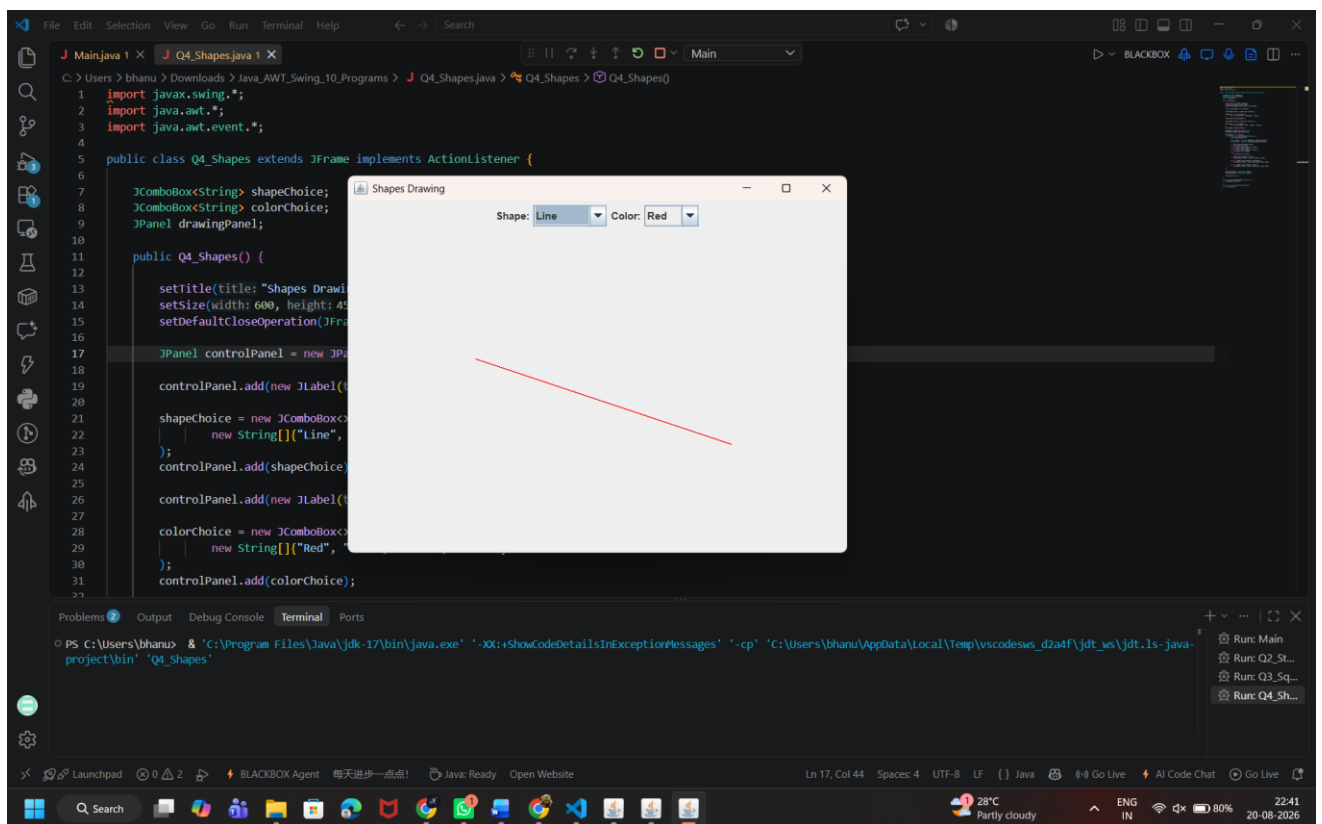
        if (shape.equals("Line"))
            g.drawLine(100, 150, 400, 150);

        else if (shape.equals("Rectangle"))
            g.drawRect(150, 100, 250, 150);

        else if (shape.equals("Oval"))
            g.drawOval(150, 100, 250, 150);
    }
}

```

Output:



10. Design a GUI application that allows the user to maintain a simple student record (add /update/delete/display) using appropriate layout manager, controls, and event handling. Code:

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class Q10_StudentRecordSystem extends JFrame {
    JTextField nameField = new JTextField();
    JTextField rollField = new JTextField();
    JTextField courseField = new JTextField();
    DefaultTableModel model = new DefaultTableModel(
        new String[]{"Name", "Roll No", "Course"}, 0);
    JTable table = new JTable(model);

    Q10_StudentRecordSystem() {
        setTitle("Student Record Management System");
        setSize(750, 500);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JPanel form = new JPanel(new GridLayout(2, 6, 10, 10));
        JButton add = new JButton("Add");
        JButton update = new JButton("Update");
        JButton delete = new JButton("Delete");

        form.add(new JLabel("Name")); form.add(nameField);
        form.add(new JLabel("Roll No")); form.add(rollField);
        form.add(new JLabel("Course")); form.add(courseField);
        form.add(add); form.add(update); form.add(delete);

        add(form, BorderLayout.NORTH);
        add(new JScrollPane(table), BorderLayout.CENTER);

        add.addActionListener(e -> addStudent());
        update.addActionListener(e -> updateStudent());
        delete.addActionListener(e -> deleteStudent());

        table.getSelectionModel().addListSelectionListener(e -> {
            int row = table.getSelectedRow();
            if (row != -1) {
                nameField.setText(model.getValueAt(row, 0).toString());
                rollField.setText(model.getValueAt(row, 1).toString());
                courseField.setText(model.getValueAt(row, 2).toString());
            }
        });
    }

    void addStudent() {
        if (nameField.getText().isEmpty() || rollField.getText().isEmpty() || courseField.getText().isEmpty())
            JOptionPane.showMessageDialog(this, "All fields are required.", "Error", JOptionPane.ERROR_MESSAGE);
        else {
            model.addRow(new Object[]{nameField.getText(), rollField.getText(), courseField.getText()});
        }
    }

    void updateStudent() {
        if (nameField.getText().isEmpty() || rollField.getText().isEmpty() || courseField.getText().isEmpty())
            JOptionPane.showMessageDialog(this, "All fields are required.", "Error", JOptionPane.ERROR_MESSAGE);
        else {
            int row = table.getSelectedRow();
            if (row != -1) {
                model.setValueAt(nameField.getText(), row, 0);
                model.setValueAt(rollField.getText(), row, 1);
                model.setValueAt(courseField.getText(), row, 2);
            }
        }
    }

    void deleteStudent() {
        int row = table.getSelectedRow();
        if (row != -1) {
            model.removeRow(row);
        }
    }
}
```

```

        || courseField.getText().isEmpty()) {
    JOptionPane.showMessageDialog(this, "Enter all details");
    return;
}

model.addRow(new Object[]{
    nameField.getText(), rollField.getText(), courseField.getText()
});
clearFields();
}

void updateStudent() {
    int row = table.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Select a student");
        return;
    }

    model.setValueAt(nameField.getText(), row, 0);
    model.setValueAt(rollField.getText(), row, 1);
    model.setValueAt(courseField.getText(), row, 2);
    JOptionPane.showMessageDialog(this, "Student Updated");
}

void deleteStudent() {
    int row = table.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Select a student");
        return;
    }
    model.removeRow(row);
    clearFields();
}

void clearFields() {
    nameField.setText("");
    rollField.setText("");
    courseField.setText("");
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new Q10_StudentRecordSystem().setVisible(true));
}
}

```

Output:

